Randomized Descent Direction in Poorly Conditioned Problems

Ali Allail KAUST
Maryam Almaskin KAUST

1 Introduction

Optimizing poorly conditioned problems, especially those characterized by high condition numbers, presents significant challenges in numerical optimization. Understanding and addressing these issues is crucial for developing effective optimization algorithms, particularly in complex domains such as machine learning, engineering, and financial modeling.
In this project, we investigate our proposed method, provide theoretical guarantees, and show our experiments in comparison to gradient descent. In low dimensional spaces (<10), our method consistently and significantly outperforms vanilla gradient descent, but as we increase the dimensionality of the problem, our method begins to perform worse and worse. Please look into the Experimental results section for more analysis.

2 Methodology

Our proposed method modifies the traditional gradient descent algorithm by incorporating a randomized component in the descent direction, aiming to mitigate the effects of high condition numbers. The steps are as follows:

1. **Gradient Calculation:** Compute the gradient, $\nabla f(x)$, at the current point x.
2. **Randomized Direction Selection:**
   This step is crucial in diverging from the path dictated by the standard gradient descent methodology. It introduces a randomized element to the direction of descent, aiming to explore the optimization landscape more broadly and potentially bypass the slow convergence issues related to poor conditioning. Two strategies are proposed for selecting this randomized direction:
   (a) **Randomization in all directions:** In this method, we generate a random vector, $u \sim U$, from some distribution U. Then, we add this random vector to the negative gradient direction as:
   $$d = -\nabla f(x) + \eta \cdot u,$$
   where η is a hyperparameter scaling factor to control the randomness.
   (b) **Randomization in a select number of directions:** Like the previous method, we generate a random vector, $u \sim U$, from some distribution U. The difference here, however, is that this vector is 0 everywhere except a select number of dimensions we want to randomize. The selected dimensions may be different at each step.
   The intuition behind this method is that in very high dimensions, the likelihood of 'luckily' getting a good descent direction randomly decreases exponentially (order 2 for each dimension). This will be discussed in the final results of the project.
   (c) **Descent Direction Validation:** In our formulation, we want to guarantee a descent direction in each step. This can be easily done by ensuring:
   $$d \cdot -\nabla f > 0$$

if 'd' does not satisfy this, then we simply multiply it by -1, and we get a descent direction.

Both approaches introduce a layer of stochasticity intended to augment the standard optimization trajectory with a mechanism to potentially overcome limitations associated with the gradient descent's susceptibility to the condition number. The efficacy of these methods, particularly in landscapes characterized by high condition numbers, forms the core of the investigative premise of this project.

3. **Backtracking Line Search:** We use backtracking line while satisfying the minimum descent condition:

$$f(x + td) < f(x) + \alpha t \nabla f(x)^T d,$$

Satisfying this ensures eventual convergence, where convergence is defined as a point with $|\nabla f < tol|$ for any $tol > 0$

The algorithms in pseudocode can be found on page **??**.

3 Experiments

In this section, we describe our experimental setup, the experiments, and asses the results.

We begin by testing our algorithm using the Rosenbrock function in 2-d:

Experiment 1

$$5(y - x^2)^2 + (1 - x)^2, \quad \kappa = 133$$

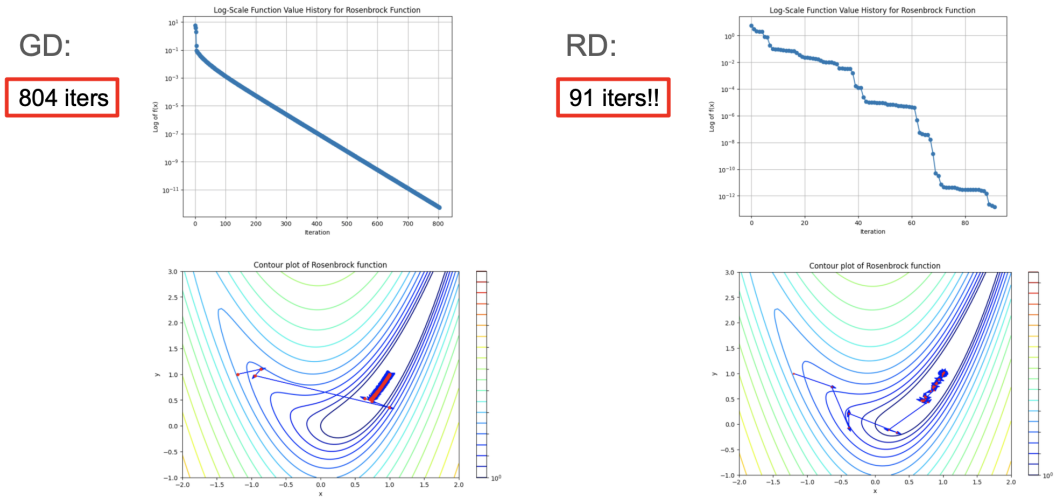


Figure 1: Experiment 1: The left side is using Gradient Descent and the right side is using the randomized descent method

It is a non-convex function used to evaluate the performance of optimization algorithms. We compare the results of our methods with those of vanilla gradient descent as shown in Fig. 1. We notice that gradient descent takes 804 iterations to converge, while our method takes only 91 iterations! At some iterations, we get lucky with the random direction, and the objective function decreases rapidly. In this example, κ is 133, which is not particularly high.
In the next experiment, we try our method using a much higher condition number.

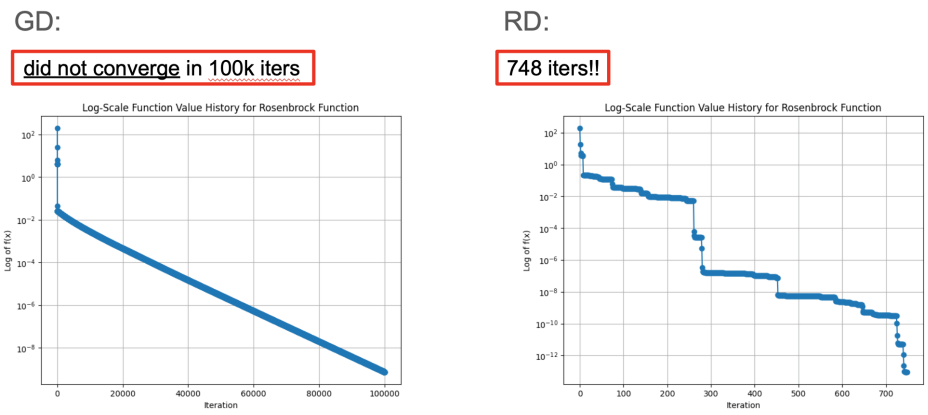


Figure 2: Experiment 1: The left side is using Gradient Descent and the right side is using the randomized descent method

Experiment 2

Let the function be:

$$1000(y - x^2)^2 + (1 - x)^2, \quad \kappa = 25008$$

Evidently, despite approximately 100,000 iterations, gradient descent failed to converge. Conversely, our randomized method achieved convergence within 748 rounds, as shown in Fig. 2.

Experiment 3

While our randomized method yielded favorable outcomes, its performance gradually diminished when increasing dimensions. Our exploration extended from 2 to 100 dimensions, with the Rosenbrock function formulated as:

$$\sum_{i=1}^{99} \left[100(x_{i+1} - x_i^2)^2 + (1 - x_i)^2\right]$$

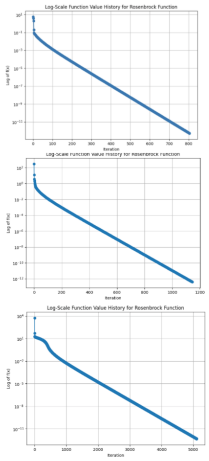
As depicted in Fig. 3, beyond five dimensions, the randomized method exhibited comparable performance to gradient descent. Notably, by the 20th dimension, our method's efficacy decreased in comparison with gradient descent. We think this happens because as we increase the dimensions, with each dimension, we decrease our chance of randomly picking the correct direction (in all dimensions) by a factor of two. We call it a 'correct' descent direction here in the sense that we choose the correct sign (positive or negative) for all dimensions. To explain this intuitively, if we were minimizing a function in 1-d space, when picking a random direction, we can either go the the right or to the left (on the number line). When we move up to 2-d, we choose to go right or left, up or down. So now, we have to choose between 4 combinations (positive or negative for each of the 2 dimensions). The same logic applies as we move to higher dimensions.
In the following experiments, we attempt different methods to mitigate this problem.

Experiment 4

In this experiment, we tried to randomize along 10 (dim/10) randomly selected dimensions. In this and the following experiments, we simplified the function to: $\sum_{i=1}^{99} \left[10(x_{i+1} - x_i^2)^2 + (1 - x_i)^2\right]$ ($\kappa = 373$).
Our method took 5407 iterations, while gradient descent took 3474 iterations.
We conclude that this method does not work.

Experiment 5

In this experiment, we selected the dimensions with the 10 smallest values in the gradient.
Our method took 5508 iterations, while gradient descent took 3474 iterations.



Dim	GD	RD
2	804	91
5	1144	764
10	2354	2248
20	5105	6190
50	14898	19043
100	37694	56807

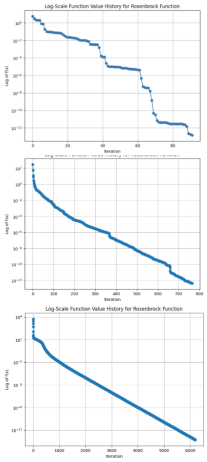


Figure 3: Experiment 1: The left side is using Gradient Descent and the right side is using the randomized descent method

This is almost no different than the previous experiment.
The premise of this method is that we have a good heuristic in selecting the dimensions to randomize. Based on the results, selecting the 10 dimensions with the smallest value does not seem to be any different than selecting 10 random directions.

Experiment 6

In this experiment, we tried a different approach: we tried an approach where we calculate the variance of each dimensions, and we randomized based on this.
First, we tried to randomize along the 10 dimensions with the highest variance. The method took 5414 iterations, so again, this does not seem to be significantly different.
Then, we also tried to randomize along the 10 dimensions with the lowest variance. The method took 5020 iterations, which is an improvement over the previous methods, but this is still worse than gradient descent which took 3474 iterations.

Experiment 7

In this experiment, we compare eight different randomization methods to evaluate the efficiency of our proposed randomized descent approach. Each method involves variations in the sampling of the random vector $\mathbf{u}$, which is either drawn from a Uniform$[-1, 1]$ distribution or a standard Normal distribution. Furthermore, we consider scenarios where $\mathbf{u}$ and the gradient $\mathbf{g}$ are either normalized or not. Because our method is random, we conducted the experiment over 100 trials and took the average.

The goal was to identify the most efficient combination of settings that minimize the average iterations. The methods were tested on the 2-d Rosenbrock function: $10000(y - x^2)^2 + (1 - x)^2$

The best performing method utilized a normalized $\mathbf{u}$ sampled from a Uniform$[-1, 1]$ distribution with a normalized gradient $\mathbf{g}$, achieving an average of 2952.75 iterations. Below we present the results from all tested combinations:

- Uniform$[-1, 1]$, $\mathbf{u}$ not normalized, $\mathbf{g}$ normalized: 4063.55 iterations.
- Uniform$[-1, 1]$, $\mathbf{u}$ not normalized, $\mathbf{g}$ not normalized: 3100.03 iterations.
- Uniform$[-1, 1]$, $\mathbf{u}$ normalized, $\mathbf{g}$ normalized: 2952.75 iterations.
- Uniform$[-1, 1]$, $\mathbf{u}$ normalized, $\mathbf{g}$ not normalized: 3145.34 iterations.
- Normal$[0, 1]$, $\mathbf{u}$ not normalized, $\mathbf{g}$ normalized: 3370.39 iterations.

- Normal$[0, 1]$, $\mathbf{u}$ not normalized, $\mathbf{g}$ not normalized: 3024.18 iterations.
- Normal$[0, 1]$, $\mathbf{u}$ normalized, $\mathbf{g}$ normalized: 2961.24 iterations.
- Normal$[0, 1]$, $\mathbf{u}$ normalized, $\mathbf{g}$ not normalized: 2985.61 iterations.

Discussion

The results indicate that normalizing both $\mathbf{u}$ and $\mathbf{g}$ consistently leads to fewer iterations on average, suggesting that normalization plays a crucial role in the efficiency of the descent process in our randomization methods because they allow us to control the magnitude of randomness in each step.

4 Conclusion

Through comparative analysis, we demonstrated the efficacy of our proposed method against the traditional gradient descent approach. Our method showcased remarkable efficiency, requiring significantly fewer iterations to converge, particularly evident in lower condition numbers.
However, we also identified limitations as the dimensionality increased. While our method initially outperformed gradient descent, its effectiveness diminished in higher dimensions. This suggests that the randomized approach may not scale well with increasing dimensionality, resulting in gradual performance degradation compared to gradient descent.
While our results are not extraordinary, we hope that our approach provides inspiration for potential solutions to poor conditioning.

References

[1] Zhang, Lijun, Mehrdad Mahdavi, and Rong Jin. "Linear convergence with condition number independent access of full gradients." Advances in Neural Information Processing Systems 26 (2013).

[2] Qu, Zheng, and Peter Richtárik. "Coordinate descent with arbitrary sampling I: Algorithms and complexity." Optimization Methods and Software 31.5 (2016): 829-857.

[3] Kingma, Diederik P., and Jimmy Ba. "Adam: A method for stochastic optimization." arXiv preprint arXiv:1412.6980 (2014).

[4] Qu, Zheng, and Peter Richtárik. "Coordinate descent with arbitrary sampling II: Expected separable overapproximation." Optimization Methods and Software 31.5 (2016): 858-884.